1 · Form ve düğme özellikleri

İki tür satır var. **Özellik:** `nesne.Özellik = değer;` (parantez yok, = var). **Metot:** `nesne.Metot();` (parantez var, = yok). Formun kendisi `this`. Satırlar olayın `{ }` arasına yazılır.

```
private void button1_Click(object sender, EventArgs e)
{
    this.Text = "Mavi";                 // ÖZELLİK: başlık
    this.BackColor = Color.Blue;        // arka renk
    this.Width = 400;                   // genişlik
    this.Height = 300;                  // yükseklik
    button1.Text = "Çalıştır";          // düğmenin yazısı
    button1.BackColor = Color.Lime;     // düğmenin rengi
    button1.Visible = false;            // görünmez
    button1.Enabled = false;            // görünür, tıklanamaz
    label1.Text = "Sonuç";              // etikete yaz
    textBox1.Text = "";                 // kutuyu temizle
    textBox1.Focus();                   // METOT: imleci kutuya koy
    MessageBox.Show("ilk program");     // METOT: mesaj penceresi
    this.Close();                       // METOT: formu kapat
}
private void Form1_Load(object sender, EventArgs e)
{
    this.BackColor = Color.Yellow;      // açılışta 1 kez çalışır
}
```

Yazı `" "` içinde · renk `Color.` ile · `true / false` tırnaksız · metot sonunda `()` · her satır `;` ile biter.

2 · Sayı alma, hesaplama, yazdırma

TextBox'taki yazı hep **metindir**: `Convert` ile sayıya çevir → hesapla → `.ToString()` ile metne çevirip yaz. Harf girilirse program çöker; bu yüzden hepsi `try { }` içine, hata olursa `catch { }` çalışır.

- **Tipler:** `int` tam sayı · `double` ondalık · `float` ondalık (çizim) · `string` metin `"…"` · `bool` true/false · `char` tek harf `'a'` · `decimal` (`decimal.MinValue`, `decimal.MaxValue`)

```
private void button1_Click(object sender, EventArgs e)
{
    try
    {
        int a;                                  // TANIMLA
        double b, sonuc;
        a = Convert.ToInt32(textBox1.Text);     // metin → tam sayı
        b = Convert.ToDouble(textBox2.Text);    // metin → ondalık
        sonuc = 2 * a + 4 * b - 1;              // HESAPLA: + - * / ( )
        label1.Text = sonuc.ToString();         // sayı → metin, YAZ
        label1.Text = Convert.ToString(sonuc);  // aynısı, uzun yolu
        label1.Text = string.Format("Toplam={0:f2}", sonuc);
        label1.Text = "Sonuç=" + sonuc.ToString();  // birleştir
        label1.Text += " devam";                // üstüne ekle
    }
    catch (Exception)
    {
        MessageBox.Show("Lütfen sayı giriniz");
        textBox1.Text = "";                     // kutuyu temizle
        textBox1.Focus();                       // imleci kutuya koy
        label1.Text = "Sonuç";
    }
}
```

- **Format:** `{0}` 1. değer, `{1}` 2. değer · `:f2` virgülden sonra 2 basamak (yalnız görüntü) · `\n` alt satır.

```
string s = string.Format("T={0:f1}\nF={1:f3}", toplam, fark);
```

TextBox'a virgülle (3,5), kodda noktayla (3.5) · `int / int` küsuratı atar: `7 / 2 = 3` · `Convert.ToInt32(textBox1.Text) = a;` yanlış.

3 · Karar yapıları: if, bool, operatörler

Şart doğruysa `{ }` içi çalışır. else-if zincirinde **yalnız ilk doğru dal** çalışır, gerisine bakılmaz. Tek satırsa `{ }` yazılmayabilir.

```
if (a > 0)
{
    label2.Text = "Sayı pozitiftir";
}
else if (a == 0)
{
    label2.Text = "Sayı sıfırdır";
}
else
{
    label2.Text = "Sayı negatiftir";
}
if (sonuc < 2000) this.BackColor = Color.Green;   // tek satır
bool sag = true;                    // bool: true ya da false
if (sag) n += 5;                    // if (sag == true) ile aynı
if (!sag) n -= 5;                   // ! = değil (sag false ise)
if (timer1.Enabled) button1.Text = "Durdur";
```

- **Karşılaştırma:** `==` eşit · `!=` eşit değil · `>` `<` `>=` `<=`
- **Mantık:** `&&` ve (ikisi de doğru) · `||` veya (biri doğru) · `!` değil
- **Kalan:** `%` → `10 % 3 = 1` · **çift:** `n % 2 == 0` · **tek:** `n % 2 == 1`

`=` atama, `==` karşılaştırma · `&&` önce, `||` sonra: `a>=7 && c<=16 || a+c<47` = `(a>=7 && c<=16) || (a+c<47)`

4 · Döngüler

for: sayısı belli tekrar. **while:** şart doğru oldukça (baştan yanlışsa hiç girmez). **do-while:** önce çalışır sonra bakar, **en az 1 kez**.

```
int toplam = 0;                     // toplam 0'dan başlar
int carpim = 1;                     // çarpım 1'den başlar
for (int i = 0; i <= 10; i++)       // başla; şart; artış
{
    toplam += i;                    // toplam = toplam + i
}
for (int i = 10; i >= 2; i -= 2)    // geriye, 2'şer
{
    carpim *= i;
}
while (a <= 3)                      // şart doğru oldukça
{
    a++;                            // a = a + 1
}
do
{
    a += 5;
} while (a == 20);                  // sonunda ; var!
while (true)
{
    k++;
    if (k == 10) break;             // break: döngüden çık
}
```

- **Atama kısaltmaları:** `x += y` (x=x+y) · `x -= y` · `x *= y` · `x /= y` · `x %= y` · `x++` · `x--`
- `return;` olaydan çıkar, alttaki kodlar çalışmaz · şartı hiç bozulmayan `while` **sonsuz döngü**.
- **Tur sayısı:** `for (i = a; i <= b; i += s)` → **(b − a) / s + 1 tur** · `<` ise b dahil değil.

"Sonuç ne olur" sorusu: tablo çiz (i | değişkenler). Her tur: şarta bak → gövdeyi yap → artışı yap. Şart yanlış olunca dur, son satırdaki değerler sonuç.

5 · Math formülleri

Hepsi `Math.` ile başlar, sonucu `double` verir. Açılar **radyan**: derece × `Math.PI / 180`.

```
double x = Math.PI;                 // π = 3,14159…
double e1 = Math.E;                 // e = 2,718…
double s = Math.Sin(aci * Math.PI / 180);   // derece → radyan
double c = Math.Cos(teta);          // radyan alır
double t = Math.Tan(teta);
double derece = teta * 180 / Math.PI;       // radyan → derece
double p = Math.Pow(taban, us); int q = Convert.ToInt32(Math.Pow(2, i)); // int'e Convert
double k = Math.Sqrt(x);            // karekök
double m = Math.Abs(x);             // mutlak değer
double y = Math.Round(x);           // en yakın tam sayı
double y2 = Math.Round(x, 2);       // virgülden sonra 2 basamak
double f = Math.Floor(x);           // aşağı yuvarla
double ta = Math.Ceiling(x);        // yukarı yuvarla
double l = Math.Log10(x);           // 10 tabanında log
double ln = Math.Log(x);            // ln
double lb = Math.Log(x, 2);         // 2 tabanında log
double ex = Math.Exp(x);            // e üssü x
double b = Math.Max(a, c);          // büyüğü
double kc = Math.Min(a, c);         // küçüğü
double aci2 = Math.Atan2(y, x);     // açı (radyan)
double aci3 = Math.Acos(x);         // ters kosinüs
double d = Math.Sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2)); // 2 nokta uzaklığı
```

- **Çevre/alan:** çember çevresi `2 * Math.PI * r` · alanı `Math.PI * r * r` · kare çevresi `4 * kenar` · alanı `kenar * kenar` · küre hacmi `4.0 / 3 * Math.PI * Math.Pow(r, 3)`
- **Örnek değerler:** `Round(17.45629, 3)=17.456` · `Floor(17.45)=17` · `Round(3.75, 1)=3.8` · `Log10(1000)=3` · `Pow(9, 3)=729` · `Ceiling(10.3)=11`

6 · Fonksiyon yazma

Tekrar eden işi bir kez yazıp adıyla çağırırsın. **Olayların dışına, class'ın içine** yazılır. `return` değeri geri verir; `void` hiçbir şey döndürmez.

```
private double KareninAlani(double kenar)       // DÖNÜŞ TİPİ: double
{
    return kenar * kenar;
}
private string Kontrol(int sayi)                // DÖNÜŞ TİPİ: string
{
    string sonuc;
    if (sayi > 0) sonuc = "Sayı pozitiftir";
    else if (sayi == 0) sonuc = "Sayı sıfırdır";
    else sonuc = "Sayı negatiftir";
    return sonuc;
}
private int Hesapla(int a, int b, int c, int d) // birden çok argüman
{
    return (a * c) + (b * d);
}
private void mb(string s)                       // void: döndürmez
{
    MessageBox.Show(s);
}
private float cs(double d)                      // çizim için kısaltma
{
    return Convert.ToSingle(d);
}
```

- **Çağırma (olayın içinde):** `alan = KareninAlani(kenar);` · `label6.Text = Kontrol(sayi);` · `mb("Lütfen sayı giriniz");`

7 · Liste (List) ve ListBox

List boyutu belli olmayan sayı dizisidir; **global** tanımlanır. ListBox ekranda görünen listedir. Sıra numarası (indeks) **0'dan** başlar: 1. eleman `[0]`.

```
List<int> liste = new List<int>();   // global (List<double>, List<string> de olur)
int k = 0;                           // global: eleman sayısı
liste.Add(sayi);                     // listeye ekle
listBox1.Items.Add(string.Format("{0}. Sayı={1}", k + 1, sayi));
liste.RemoveAt(liste.Count - 1);     // SON elemanı sil
liste.RemoveAt(index - 1);           // n. elemanı sil
liste.Clear();                       // listeyi boşalt
listBox1.Items.Clear();              // kutuyu boşalt
int deger = liste[index - 1];        // n. elemanı al
int son = liste[liste.Count - 1];    // son eleman
listBox1.SelectedIndex = index - 1;  // kutuda seç
if (liste.Count > 0) { … }           // liste boş değilse
for (int i = 0; i < liste.Count; i++)   // listeyi gez
{
    if (liste[i] % 2 == 1) tekToplam += liste[i];
    if (liste[i] % 2 == 0) ciftCarpim *= liste[i];
    listBox1.Items.Add(string.Format("{0}. Sayı={1}", i + 1, liste[i]));
}
```

- **Silince:** `listBox1.Items.Clear();` → yukarıdaki `for` ile yeniden doldur → `k--;`
- **Yüzde:** `double oran = Convert.ToDouble(gecen) / Convert.ToDouble(liste.Count) * 100;`
- **Dizi:** `double[] sayi = new double[1000];` · **ArrayList:** `using System.Collections;` + `ArrayList arb = new ArrayList();`
- **Geçersiz sıra:** `if (index > liste.Count) { MessageBox.Show("…"); return; }`

11 · Diğer: kodlarda geçenler

```
using System.Threading;                 // Thread.Sleep için
Thread.Sleep(1000);                     // 1000 ms bekle
Application.DoEvents();                 // döngüde form donmasın
Random ra = new Random();               // global
int sayi = ra.Next(0, 600);             // 0-599 arası rastgele
int saat = DateTime.Now.Hour;           // .Minute, .Second
private void radioButton1_CheckedChanged(object sender, EventArgs e)
{
    if (radioButton1.Checked) this.Text = "1. seçildi";
}
```

8 · Çizim (Paint)

Çizim kodu sadece `Form1_Paint` olayına yazılır. `e.Graphics` ile çizilir. Çizimi yenilemek için başka olaydan `Invalidate();` çağrılır.

```
private void Form1_Paint(object sender, PaintEventArgs e)
{
    e.Graphics.TranslateTransform(this.Width / 2, this.Height / 2);
    Pen p = new Pen(Color.Red, 3);              // renk, kalınlık
    e.Graphics.DrawLine(p, x1, y1, x2, y2);     // çizgi: 2 nokta
    e.Graphics.DrawRectangle(p, x, y, gen, yuk);    // dikdörtgen
    e.Graphics.FillRectangle(Brushes.DodgerBlue, x, y, gen, yuk);
    e.Graphics.DrawEllipse(p, x, y, gen, yuk);  // elips / daire
    e.Graphics.FillEllipse(Brushes.Red, x, y, gen, yuk);
    e.Graphics.DrawLine(Pens.Black, 0, 0, 300, 300);  // hazır kalem
}
```

- **TranslateTransform(a, b):** (0,0) noktasını (a, b)'ye taşır; `this.Width / 2, this.Height / 2` = formun ortası.
- **Eksen:** x sağa, **y aşağı** artar. Yukarıdaki nokta = **eksi y**.
- **Dikdörtgen/elips:** (x, y) **sol üst köşe**, sonra genişlik, yükseklik. Sol **alt** köşe verilirse y yerine `y - yuk`.
- **Merkezi (cx, cy), yarıçapı r daire:** `DrawEllipse(p, cx - r, cy - r, 2 * r, 2 * r)`
- **Draw:** içi boş, `Pen` alır · **Fill:** içi dolu, `Brushes.Renk` ya da `SolidBrush` alır.

double değişkenle çizerken `Convert.ToSingle(x)` ya da `cs(x)` şart · Kâğıtta önce (0,0)'ı seç, her köşenin koordinatını şekle yaz.

9 · Timer ve animasyon

Timer: `Interval` ms'de bir `timer1_Tick` çalışır. Tick'te konum değişir, `Invalidate();` Paint'i yeniden çalıştırır → şekil kayar. Paint'te hareketli parçanın koordinatına `n` eklenir.

```
int n = 0;                          // GLOBAL: konum
bool sag = true;                    // GLOBAL: yön
private void Form1_Load(object sender, EventArgs e)
{
    this.SetStyle(ControlStyles.AllPaintingInWmPaint, true);
    this.SetStyle(ControlStyles.OptimizedDoubleBuffer, true); // titreme önler
    timer1.Interval = 10;           // 10 ms'de bir Tick
}
private void timer1_Tick(object sender, EventArgs e)
{
    if (n == 0) sag = true;         // başa geldi → sağa
    if (n == 760) sag = false;      // sona geldi → sola
    if (sag) n += 5;
    else n -= 20;                   // 760, 5'e ve 20'ye tam bölünür
    label1.Text = string.Format("sağ={0} : n={1}", sag, n);
    Invalidate();                   // Paint'i çağır
}
private void button1_Click(object sender, EventArgs e)
{
    timer1.Enabled = !timer1.Enabled;   // çalışıyorsa durdur, duruyorsa çalıştır
    if (timer1.Enabled) { button1.Text = "Durdur"; button1.BackColor = Color.Red; }
    else { button1.Text = "Çalıştır"; button1.BackColor = Color.Lime; }
}
private void button2_Click(object sender, EventArgs e)    // başa al
{
    n = 0; Invalidate(); timer1.Enabled = false;
    button1.Text = "Çalıştır"; button1.BackColor = Color.Lime;
}
private void Form1_Resize(object sender, EventArgs e)
{
    Invalidate();                   // form boyutu değişince yeniden çiz
}
private void hScrollBar1_Scroll(object sender, ScrollEventArgs e)
{
    timer1.Interval = hScrollBar1.Value;    // hız ayarı (Interval 0 olamaz → Minimum 1)
    label2.Text = hScrollBar1.Value.ToString();
}
```

- **Paint'te hareketli parça:** `e.Graphics.FillRectangle(Brushes.DodgerBlue, -500 + n, -40, 40, 40);`
- **Bağlı parçalar (ip, makara):** aynı `n` hepsine: üst kütle `-200 + n` (sağa), alt kütle `0 + n` (aşağı), ip uçları da `+ n`.
- **Durdurma:** `if (n == 160) { timer1.Enabled = false; button1.Enabled = false; this.Text = "bitti"; }`

"X ms'de 1 piksel" → `Interval = X;` + Tick'te `n++;` · "butona basınca duruyorsa çalışsın" → `timer1.Enabled = !timer1.Enabled;`

10 · Dönme hareketi

Çember üstündeki nokta: `x = L * Math.Cos(teta)`, `y = L * Math.Sin(teta)`. Tick'te `teta` artar, nokta döner. `teta +=` **saat yönü**, `teta -=` saat tersi.

```
double L = 200, teta = 0, x = 200, y = 0;   // GLOBAL
double r = 20;                              // topun yarıçapı
int n = 1, k = 5;                           // derece/tick, kalınlık
Color c = Color.Blue;
SolidBrush b = new SolidBrush(Color.Red);   // rengi değişebilir fırça
private void Form1_Paint(object sender, PaintEventArgs e)
{
    e.Graphics.TranslateTransform(this.Width / 2, this.Height / 2);
    Pen pBlue = new Pen(c, k);
    e.Graphics.DrawLine(pBlue, 0, 0, cs(x), cs(y));                 // kol
    e.Graphics.DrawEllipse(pBlue, cs(-L), cs(-L), cs(2 * L), cs(2 * L));
    e.Graphics.FillEllipse(b, cs(x - r), cs(y - r), cs(2 * r), cs(2 * r));
}
private void timer1_Tick(object sender, EventArgs e)
{
    teta += n * Math.PI / 180;              // n derece ilerle
    x = L * Math.Cos(teta); y = L * Math.Sin(teta);
    double tetaDeg = Math.Round(teta * (180 / Math.PI));  // toplam derece
    double tetaDeg2 = tetaDeg % 360;        // 0-360 arası
    double turSayisi = tetaDeg / 360;
    label1.Text = string.Format("Açı={0} Derece : Tur Sayısı={1:f2}",
                                tetaDeg2, turSayisi);
    Invalidate();
}
private void button3_Click(object sender, EventArgs e)      // renk seç
{
    if (colorDialog1.ShowDialog() == DialogResult.OK)
    { c = colorDialog1.Color; Invalidate(); }
    if (colorDialog2.ShowDialog() == DialogResult.OK)       // top rengi
    { Color c2 = colorDialog2.Color; b = new SolidBrush(c2); Invalidate(); }
}
private void timer2_Tick(object sender, EventArgs e)        // hızlanma
{
    timer2.Interval = 1000;
    if (n == 20) return;                    // 20'de dur, alttakiler çalışmaz
    n++; hScrollBar2.Value = n; Invalidate();
}
private void hScrollBar1_Scroll(object sender, ScrollEventArgs e)
{
    L = hScrollBar1.Value; label2.Text = string.Format("L={0}", L); Invalidate();
}
```

- **Başlangıç yukarıda:** `teta = -Math.PI / 2` · **dur:** `if (teta >= Math.PI / 2) timer1.Enabled = false;`
- **Kare köşeleri:** 4 açı 90° arayla (`-Math.PI / 2, 0, Math.PI / 2, Math.PI`), hepsine aynı artış.
- **Yuvarlanma:** `a += r * Math.PI / 180;` (öteleme = r × açı) · daire merkezi `(a, 0)` · 1 tur = `2 * Math.PI * r`
- **İp boyu sabit (Pisagor):** `b = 300 - Math.Sqrt(dx * dx + dy * dy);`

12 · Sık hatalar

- `int / int` küsuratı atar: `4 / 3 = 1` → `4.0 / 3`
- Sin/Cos **radyan** ister: `aci * Math.PI / 180`
- Liste **0'dan**: 7. eleman `[6]`, son eleman `[Count - 1]`
- TextBox metindir: önce `Convert`, yazarken `.ToString()`
- Çarpım `1`'den, toplam `0`'dan başlar · do-while en az 1 kez çalışır
- Global değişkeni olayın içinde **tekrar tanımlama** (`int n = 0;` her tıkta sıfırlar)
- `Invalidate();` yazılmazsa ekran değişmez · Draw'da double → `cs(x)`
- `catch` hemen `try`'ın `}`'sinden sonra gelir · `do { } while (…);` sonunda `;`

14 · Eski vizelerde gerekenler

```
x = cx + r * Math.Cos(teta); y = cy + r * Math.Sin(teta);  // merkezi (cx,cy) dönme
teta1 += Math.PI / 45; teta2 += Math.PI / 90;   // farklı hız: 4°, 2° (kasnak)
for (int i = 1; i <= m; i++)                     // iç içe for: Σ Σ (i+j)^x
{ for (int j = 2; j <= n; j++) { toplam += Math.Pow(i + j, x); } }
double delta = b * b - 4 * a * c;               // 2. derece denklem
double k1 = (-b + Math.Sqrt(delta)) / (2 * a);  // kök 2: -b - Math.Sqrt…
if (delta < 0) return 0;          // fonksiyonda: orada biter, alttakiler çalışmaz
 // üçgen = 3 DrawLine: köşe1 → köşe2 → köşe3 → köşe1
```

- **Hesapla + çiz:** değişken **global** → Click'te hesapla → `Invalidate();` → Paint'te çiz.

13 · İskelet: neyi nereye yazacağım

Sıra: **using** → **namespace** → **class** → ① **global** → ② **Form1()** → **olaylar** → ⑫ **fonksiyonlar**. Aynı renkteki `{ }` birbirini açar-kapatır. `Program.cs` (static void Main) hazır gelir, yazılmaz.

```
using System; using System.Collections.Generic; using System.ComponentModel;
using System.Data; using System.Drawing; using System.Linq;
using System.Text; using System.Windows.Forms;
using System.Collections; using System.Threading;  // sınavda hepsi ALT ALTA
namespace ad_numara                     // proje adı
{
    public partial class Form1 : Form
    {
        int n = 0; bool sag = true;  // ① GLOBAL → 7, 9, 10
        public Form1()
        {
            InitializeComponent();    // ② dokunma
        }
        private void Form1_Load(object sender, EventArgs e)
        {   // ③ açılışta → 1, 9
        }
        private void button1_Click(object sender, EventArgs e)
        {
            try
            {
                if (n > 0)
                {   // ④ şart doğru → 3
                }
                else if (n == 0)
                {   // ④ ikinci şart
                }
                else
                {   // ④ hiçbiri
                }
                for (int i = 0; i < 10; i++)
                {   // ⑤ döngü → 4
                }
                while (n < 10)
                {
                    n++;
                }
                do
                {
                    n--;
                } while (n > 5);
            }
            catch (Exception)
            {   // ⑥ hata olursa → 2
            }
        }
        private void Form1_Paint(object sender, PaintEventArgs e)
        {   // ⑦ çizim → 8
        }
        private void timer1_Tick(object sender, EventArgs e)
        {
                                // ⑧ her tick → 9, 10
            Invalidate();
        }
        private void Form1_Resize(object sender, EventArgs e)
        {
            Invalidate();             // ⑨ boyut değişince
        }
        private void hScrollBar1_Scroll(object sender, ScrollEventArgs e)
        {   // ⑩ kaydırınca → 9, 10
        }
        private void radioButton1_CheckedChanged(object sender, EventArgs e)
        {   // ⑪ seçim değişince → 11
        }
        private double Kare(double x)
        {
            return x * x;             // ⑫ fonksiyon → 6
        }
        private void mb(string s)
        {
            MessageBox.Show(s);       // ⑫ void
        }
    }
}
```

ANİMASYON SORUSU NASIL KURULUR
1. Şekli kâğıda çiz, (0,0)'ı seç → `TranslateTransform`
2. Sabit parçaların köşe koordinatlarını şekle yaz
3. Hareketli parçanın başlangıcına `+ n` (dönmede `teta`)
4. İp: `b = L - Math.Sqrt(dx * dx + dy * dy)`
dx, dy = makara − ipin bağlı olduğu nokta
5. Dönme: `x = cx + r * Math.Cos(teta)` (y'de `Sin`)
6. Kayış: `DrawLine(p, cx1, -r1, cx2, -r2)` (alttaki `+r`)
7. Yuvarlanma: Tick'te `teta` ve `a` birlikte artar
8. "X ms'de Y" → `Interval = X;` Tick'te `n += Y;`
9. Bitiş konumu → `if (n == bitiş) timer1.Enabled = false;`